

MAHARISHI MARKANDESHWAR UNIVERSITY

Department of Computer Science & Engineering

A PROJECT REPORT ON

SCALABLE FRAUD DETECTION PIPELINE USING PYSPARK, SPARK SQL AND DATABRICKS

(Medallion Architecture – Bronze, Silver, Gold)

Submitted in partial fulfilment of the requirements of the

Internship Program

at

Celebal Technologies

Submitted by : Naman Garg

Roll no : 11232740

LMS id : CT_CSI_DE_67

Table of Contents

1. Introduction
2. Objective
3. Tools and Technologies Used
4. System Architecture
5. Dataset Description
6. Methodology / Implementation
 - 6.1 Bronze Layer – Raw Data Ingestion
 - 6.2 Silver Layer – Data Cleaning and Enrichment
 - 6.3 Gold Layer – Fraud Detection Logic
 - 6.4 Aggregated Insights using Spark SQL
7. Results and Analysis
8. Challenges Faced
9. Conclusion
10. Future Scope
11. References / Project Link

1. Introduction

Financial institutions process massive volumes of transactions on a daily basis, making it increasingly difficult to identify fraudulent activity manually or through simple rule checks applied on raw, unprocessed data. Poor data quality — such as missing values, inconsistent formats, and mismatched records — further complicates the task of reliably detecting fraud.

To address this, the project implements a structured, layered data pipeline capable of ingesting raw data, progressively cleaning and enriching it, and applying fraud detection logic to classify transactions. The pipeline is designed using the Medallion Architecture (Bronze → Silver → Gold), a widely adopted pattern in modern data engineering that ensures data quality improves at every stage before insights are generated.

2. Objective

- To ingest raw CSV data (accounts, transactions, and fraud watchlist) into a structured pipeline.
- To clean and enrich the data by handling missing values, correcting data types, and joining related datasets.
- To apply fraud detection logic by matching transactions against a known fraud watchlist.
- To generate aggregated, actionable insights such as total transactions, fraud counts, and account-level fraud analysis.

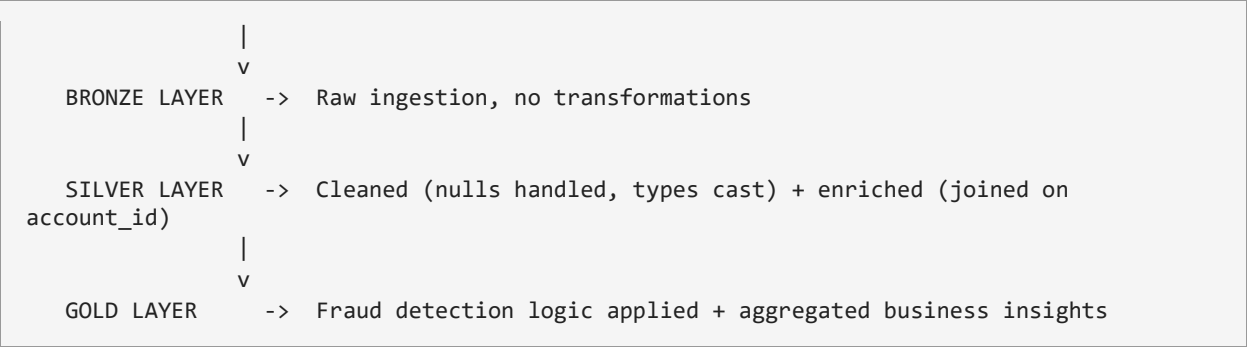
3. Tools and Technologies Used

Tool	Purpose
PySpark	Distributed data processing for large-scale transformations
Spark SQL	Writing aggregation and business reporting queries
Databricks (Free Edition)	Cloud-based notebook environment with serverless compute
Delta Lake	Storage format used for Bronze, Silver, and Gold tables

4. System Architecture

The pipeline follows the Medallion Architecture, consisting of three progressive layers:

Raw CSVs (accounts, transactions, fraud watchlist)
--



Layer	Purpose	Key Operations
Bronze	Raw data capture	Read CSVs, save as Delta tables (no transformation)
Silver	Data cleaning and enrichment	Null handling, type casting, join on account_id
Gold	Business logic and insights	Fraud classification, aggregation, reporting

5. Dataset Description

File	Key Columns	Description
accounts.csv	account_id, customer_name, account_type, opening_date, branch, kyc_status, credit_limit	Customer account master data
transactions.csv	txn_id, account_id, txn_date, txn_type, amount, merchant, city, is_international	Individual transaction records
known_fraud_accounts.csv	account_id, fraud_type, flagged_date	Watchlist of accounts previously flagged for fraud

During initial data profiling, a few real-world data quality issues were observed — the credit_limit column in accounts.csv contained missing values, the merchant column in transactions.csv had missing entries, and some account_id values in transactions.csv did not have a matching record in accounts.csv. These issues were addressed during the Silver layer processing, as described in the next section.

6. Methodology / Implementation

6.1 Bronze Layer – Raw Data Ingestion

The raw CSV files were read using Spark's DataFrame reader with schema inference and stored as Delta tables exactly as received, preserving an unaltered copy of the source data.

```
accounts_df = spark.read.csv(accounts_path, header=True, inferSchema=True)
transactions_df = spark.read.csv(transactions_path, header=True, inferSchema=True)
watchlist_df = spark.read.csv(watchlist_path, header=True, inferSchema=True)

accounts_df.write.format("delta").mode("overwrite").saveAsTable("bronze_accounts")
transactions_df.write.format("delta").mode("overwrite").saveAsTable("bronze_transactions")
watchlist_df.write.format("delta").mode("overwrite").saveAsTable("bronze_watchlist")
```

6.2 Silver Layer – Data Cleaning and Enrichment

- Dropped records with missing account_id or txn_id to maintain referential integrity.
- Cast amount and credit_limit columns to double for accurate numeric computation.
- Converted opening_date and txn_date columns to proper date format.
- Filled missing merchant values with “Unknown” and missing credit_limit values with 0.
- Performed a left join between transactions and accounts on account_id to enrich each transaction with account-level details.

```
accounts_clean = spark.table("bronze_accounts") \
    .dropna(subset=["account_id"]) \
    .withColumn("opening_date", to_date(col("opening_date"))) \
    .withColumn("credit_limit", col("credit_limit").cast("double")) \
    .fillna({"credit_limit": 0})

transactions_clean = spark.table("bronze_transactions") \
    .dropna(subset=["txn_id", "account_id"]) \
    .withColumn("txn_date", to_date(col("txn_date"))) \
    .withColumn("amount", col("amount").cast("double")) \
    .fillna({"merchant": "Unknown"})

silver_df = transactions_clean.join(accounts_clean, on="account_id", how="left")
silver_df.write.format("delta").mode("overwrite").saveAsTable("silver_transactions_enriched")
```

6.3 Gold Layer – Fraud Detection Logic

The enriched Silver dataset was joined with the fraud watchlist on account_id. Transactions belonging to a flagged account were classified as fraud, while all others were classified as normal.

```
watchlist_clean = spark.table("bronze_watchlist") \
    .dropna(subset=["account_id"]) \
    .select(col("account_id").alias("flagged_account"), "fraud_type", "flagged_date")
```

```

gold_df = silver_df.join(
    watchlist_clean,
    silver_df.account_id == watchlist_clean.flagged_account,
    how="left"
)

gold_df = gold_df.withColumn(
    "fraud_status",
    when(col("flagged_account").isNotNull(), "fraud").otherwise("normal")
)

gold_df.write.format("delta").mode("overwrite").saveAsTable("gold_fraud_transactions")

```

6.4 Aggregated Insights using Spark SQL

Two summary tables were generated from the Gold layer using Spark SQL for business-level reporting.

```

-- Overall fraud vs normal summary
SELECT fraud_status,
       COUNT(*) AS txn_count,
       ROUND(SUM(amount), 2) AS total_amount,
       ROUND(AVG(amount), 2) AS avg_amount
FROM gold_fraud_transactions
GROUP BY fraud_status

-- Account-level fraud analysis
SELECT account_id, customer_name,
       COUNT(*) AS total_txns,
       SUM(CASE WHEN fraud_status = 'fraud' THEN 1 ELSE 0 END) AS fraud_txns,
       ROUND(SUM(CASE WHEN fraud_status = 'fraud' THEN amount ELSE 0 END), 2) AS
fraud_amount
FROM gold_fraud_transactions
GROUP BY account_id, customer_name
HAVING fraud_txns > 0
ORDER BY fraud_txns DESC

```

7. Results and Analysis

The pipeline was executed end-to-end on the provided dataset (200 transactions across 56 accounts, with 10 accounts on the fraud watchlist). The key results obtained are summarized below.

Metric	Normal	Fraud
Transaction Count	174	26

Metric	Normal	Fraud
Total Amount (₹)	85,16,424.86	38,07,632.11
Average Amount (₹)	48,944.97	1,46,447.39

Key Observation

Fraudulent transactions account for only around 13% of the total transaction volume, yet their average transaction amount is nearly three times higher than that of normal transactions (₹1,46,447 vs ₹48,945). This suggests that fraudulent activity in this dataset is characterized by fewer, higher-value transactions rather than frequent, low-value ones — a pattern often observed in real-world fraud cases.

Screenshots of Pipeline Execution

The following screenshots document successful execution of each stage of the pipeline in the Databricks environment.

[Insert Screenshot: Bronze Layer – Data Ingestion]

[Insert Screenshot: Bronze Layer – Tables Saved]

[Insert Screenshot: Silver Layer – Cleaned & Enriched Data]

[Insert Screenshot: Silver Layer – Table Saved]

[Insert Screenshot: Gold Layer – Fraud Classification]

[Insert Screenshot: Gold Layer – Table Saved]

[Insert Screenshot: Fraud Summary Report]

[Insert Screenshot: Account-Level Fraud Analysis]

8. Challenges Faced

- Handling missing values in `credit_limit` and `merchant` columns without distorting downstream calculations.
- Some transactions referenced `account_id` values not present in `accounts.csv`, requiring a left join instead of an inner join to avoid losing transaction records.

- Ensuring join keys were unambiguous while combining three separate datasets in the Gold layer.
- Understanding and correctly applying the Medallion Architecture pattern rather than writing a single, monolithic transformation script.
- Learning to combine PySpark DataFrame operations with Spark SQL for cleaner, business-friendly reporting.

9. Conclusion

This project provided practical, hands-on experience in building a scalable data pipeline for fraud detection using PySpark, Spark SQL, and Databricks. By following the Medallion Architecture, the pipeline ensures that data quality improves progressively at each stage, fraud detection logic is applied consistently, and the resulting output provides clear, actionable business insights.

Through this internship project, I gained a strong practical understanding of data engineering concepts, including data ingestion, cleaning, enrichment, and aggregation at scale — concepts that are directly applicable to real-world data engineering and analytics roles.

10. Future Scope

- Replacing the binary fraud flag with a weighted risk-scoring model based on multiple factors such as transaction amount and international transaction flag.
- Incorporating streaming ingestion using Spark Structured Streaming for near real-time fraud detection.
- Adding automated data quality checks between pipeline layers.
- Integrating a machine learning model trained on historical fraud patterns for more advanced anomaly detection.

11. References / Project Link

Complete source code, execution screenshots, and documentation for this project are available at the following repository:

https://github.com/Namangarg001/Data-cleaning-assignment/tree/main/fraud_detection_pipeline